

Ex4-1-Pade-sol1

March 30, 2026

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import toeplitz, solve, pinv, inv
from scipy.signal import lfilter, freqz
```

1 Exercice 4.1 : Approximation de Padé

(M.Tognolini HEIG-VD 2026 MA-StatDig)

1.1 4.1.1. Introduction et Définition du Signal

L'objectif est de tester l'approximation de Padé sur un signal test $x(n)$. Le signal est défini sur $N = 12$ échantillons par :

$$x(n) = [1, 3 \cdot (0.5)^n] \text{ pour } n = 1..N - 1$$

```
[2]: # Paramètres du signal
N = 12
n = np.arange(N)
xn = np.zeros(N)
xn[0] = 1.0
xn[1:] = 3 * (0.5)**n[1:]

print("Signal xn (12 échantillons) :") # faire un table a 2 colonnes : n et xn
print("\n\t\txn")
for i in range(len(n)):
    print(f"{n[i]}\t\t{xn[i]}")
```

Signal xn (12 échantillons) :

n	xn
0	1.0
1	1.5
2	0.75
3	0.375
4	0.1875
5	0.09375
6	0.046875

7	0.0234375
8	0.01171875
9	0.005859375
10	0.0029296875
11	0.00146484375

1.2 4.1.2. Test a) : Modèle AR(2) ($p = 2, q = 0$)

Dans ce cas, nous cherchons un modèle purement auto-régressif. La démarche consiste à : 1. Construire la matrice de convolution X . 2. Extraire la matrice X_q et le vecteur x_{q+1} (échantillons de $q+1$ à $q+p$). 3. Résoudre le système $X_q \mathbf{a}_p = -\mathbf{x}_{q+1}$ pour trouver les coefficients du dénominateur.

Pour la matrice de convolution utiliser la fonction:

```
# Construction de la matrice de convolution (équivalent convmtx)
def get_conv_mtx(x, p_val):
    col = x
    row = np.zeros(p_val + 1)
    row[0] = x[0]
    return toeplitz(col, row)
```

```
[3]: p = 2
q = 0

# Construction de la matrice de convolution (équivalent convmtx)
def get_conv_mtx(x, p_val):
    col = x
    row = np.zeros(p_val + 1)
    row[0] = x[0]
    return toeplitz(col, row)

X_full = get_conv_mtx(xn, p)

print("Matrice de convolution X_full :")
print(X_full)
# Extraction de Xq et xq_1 pour p=2, q=0
# Xq doit être de taille p x p
Xq = X_full[q : q+p, 0:p]
xq_1 = xn[q+1 : q+p+1]

print("Matrice Xq :")
print(Xq)
print("Vecteur xq_1 :")
print(xq_1)

# Calcul des coefficients ap
# Système : Xq * ap[1:] = -xq_1
# ap_tail = solve(Xq, -xq_1)
```

```

ap_tail = inv(Xq) @ (-xq_1) # Utilisation de la pseudo-inverse pour plus de
↳ robustesse
ap = np.concatenate(([1.0], ap_tail))

# Coefficient bq (pour q=0, b0 = x(0))
bq = np.array([xn[0]])

print(f"Coefficients ap : {ap}")
print(f"Coefficients bq : {bq}")

# Reconstruction et Erreur
xhat = lfilter(bq, ap, np.eye(1, N)[0])
Els = np.linalg.norm(xn - xhat)
print(f"Erreur Els : {Els:.4f}")

# affiche la réponse impulsionnelle et le signal original ainsi que l'erreur
plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.stem(n, xn, linefmt='b-', markerfmt='bo', label='Signal original $x(n)$')
plt.stem(n, xhat, linefmt='r-', markerfmt='ro', label='Signal reconstruit
↳ ${xhat}(n)$')
plt.title("Signal Original vs Reconstruit")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()
plt.subplot(2, 1, 2)
plt.stem(n, xn - xhat, linefmt='m-', markerfmt='mo', label='Erreur $x(n) -
↳ ${xhat}(n)$')
plt.title("Erreur de Reconstruction")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()

```

Matrice de convolution X_full :

```

[[1.00000000e+00 0.00000000e+00 0.00000000e+00]
 [1.50000000e+00 1.00000000e+00 0.00000000e+00]
 [7.50000000e-01 1.50000000e+00 1.00000000e+00]
 [3.75000000e-01 7.50000000e-01 1.50000000e+00]
 [1.87500000e-01 3.75000000e-01 7.50000000e-01]
 [9.37500000e-02 1.87500000e-01 3.75000000e-01]
 [4.68750000e-02 9.37500000e-02 1.87500000e-01]
 [2.34375000e-02 4.68750000e-02 9.37500000e-02]
 [1.17187500e-02 2.34375000e-02 4.68750000e-02]

```

```
[5.85937500e-03 1.17187500e-02 2.34375000e-02]
[2.92968750e-03 5.85937500e-03 1.17187500e-02]
[1.46484375e-03 2.92968750e-03 5.85937500e-03]]
```

Matrice X_q :

```
[[1.  0. ]
 [1.5 1. ]]
```

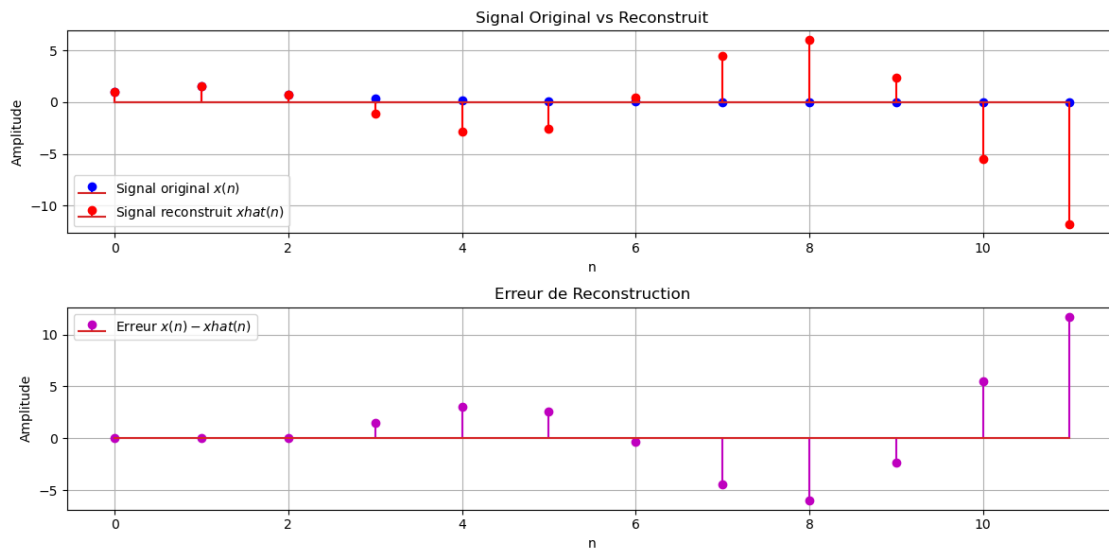
Vecteur x_{q_1} :

```
[1.5 0.75]
```

Coefficients a_p : [1. -1.5 1.5]

Coefficients b_q : [1.]

Erreur Els : 15.7206



1.3 4.1.3. Test b) : Modèle MA(2) ($p = 0, q = 2$)

Pour un modèle Moving Average, la solution est triviale car $h(n) = b_q(n)$. Les coefficients du numérateur sont simplement les $q + 1$ premiers échantillons du signal.

$$b_q = [x(0), x(1), x(2)]$$

$$a_p = [1]$$

```
[4]: p_ma = 0
q_ma = 2

bq_ma = xn[:q_ma+1]
ap_ma = np.array([1.0])

xhat_ma = lfilter(bq_ma, ap_ma, np.eye(1, N)[0])
Els_ma = np.linalg.norm(xn - xhat_ma)
```

```

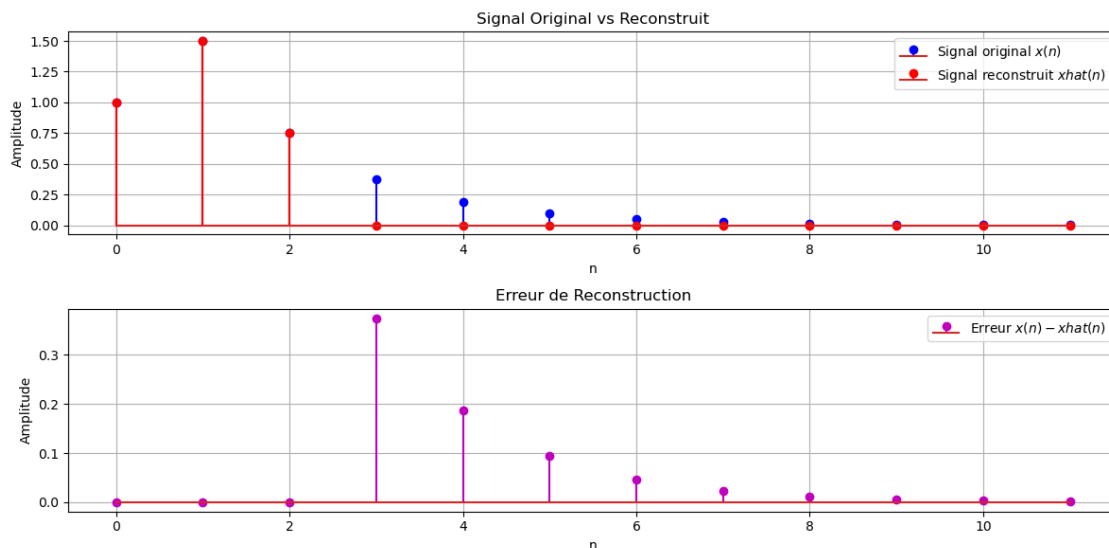
print(f"Coefficients bq (MA) : {bq_ma}")
print(f"Erreur Els MA : {Els_ma:.4f}")

# affiche la réponse impulsionnelle et le signal original ainsi que l'erreur
plt.figure(figsize=(12, 6))
plt.subplot(2, 1, 1)
plt.stem(n, xn, linefmt='b-', markerfmt='bo', label='Signal original $x(n)$')
plt.stem(n, xhat_ma, linefmt='r-', markerfmt='ro', label='Signal reconstruit_
↳ ${xhat}(n)$')
plt.title("Signal Original vs Reconstruit")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()
plt.subplot(2, 1, 2)
plt.stem(n, xn - xhat_ma, linefmt='m-', markerfmt='mo', label='Erreur $x(n) -_
↳ ${xhat}(n)$')
plt.title("Erreur de Reconstruction")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()

```

Coefficients bq (MA) : [1. 1.5 0.75]

Erreur Els MA : 0.4330



1.4 4. Test c) : Modèle ARMA(1,1) ($p = 1, q = 1$)

IIR à deux étapes : 1. Calcul de a_p via les équations de Padé décalées. 2. Calcul de b_q via la relation $b_q = X_{0:q} \cdot a_p$.

```
[17]: p_arma = 1
      q_arma = 1

      # Étape 1 : Dénominateur
      X_full_arma = get_conv_mtx(xn, p_arma)
      Xq_arma = X_full_arma[q_arma : q_arma+p_arma, 0:p_arma]
      xq_1_arma = xn[q_arma+1 : q_arma+p_arma+1]
      print("Matrice Xq pour ARMA :")
      print(Xq_arma)
      print("Vecteur xq_1 pour ARMA :")
      print(xq_1_arma)

      ap_tail_arma = solve(Xq_arma, -xq_1_arma)
      ap_arma = np.concatenate([1.0], ap_tail_arma)

      # Étape 2 : Numérateur
      # b = X(0:q, 0:p) * a

      X_top = X_full_arma[0:q_arma+1, 0:p_arma+1]
      print("Matrice X_top pour ARMA :")
      print(X_top)
      bq_arma = X_top @ ap_arma

      print(f"ap ARMA(1,1) : {ap_arma}")
      print(f"bq ARMA(1,1) : {bq_arma}")

      xhat_arma = lfilter(bq_arma, ap_arma, np.eye(1, N)[0])
      Els_arma = np.linalg.norm(xn - xhat_arma)
      print(f"Erreur Els ARMA(1,1) : {Els_arma}")

      # affiche la réponse impulsionnelle et le signal original ainsi que l'erreur
      plt.figure(figsize=(12, 6))
      plt.subplot(2, 1, 1)
      plt.stem(n, xn, linefmt='b-', markerfmt='bo', label='Signal original $x(n)$')
      plt.stem(n, xhat_arma, linefmt='r-', markerfmt='ro', label='Signal reconstruit_1  
↪ ${xhat}(n)$')
      plt.title("Signal Original vs Reconstruit")
      plt.xlabel("n")
      plt.ylabel("Amplitude")
      plt.legend()
      plt.grid()
      plt.subplot(2, 1, 2)
```

```
plt.stem(n, xn - xhat_arma, linefmt='m-', markerfmt='mo', label='Erreur  $x(n) - \hat{x}(n)$ ')
plt.title("Erreur de Reconstruction")
plt.xlabel("n")
plt.ylabel("Amplitude")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

Matrice X_q pour ARMA :

```
[[1.5]]
```

Vecteur x_{q-1} pour ARMA :

```
[0.75]
```

Matrice X_{top} pour ARMA :

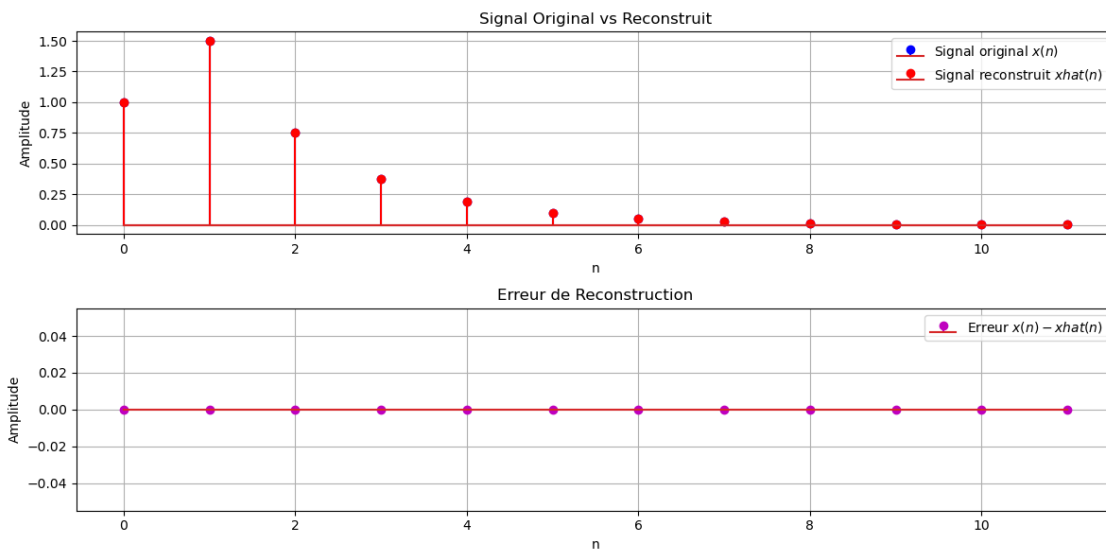
```
[[1.  0. ]
```

```
 [1.5 1.  ]]
```

ap ARMA(1,1) : [1. -0.5]

bq ARMA(1,1) : [1. 1.]

Erreur Els ARMA(1,1) : 0.0



1.5 4.1.5. Fonction Générale $\text{pade}(x, p, q)$

Ecrivez une fonction qui automatise le calcul des coefficients de Padé pour n'importe quel ordre.

Prototype :

```
def pade(x, p, q):
    N_sig = len(x)
    if p > 0:
```

```

# 1. Résoudre pour ap

# Vérification de singularité
if np.linalg.det(Xq) == 0:
    raise ValueError("Xq est une matrice singulière !")

else:
    ap = np.array([1.0])

# 2. Résoudre pour bq

# 3. Reconstruction

return ap, bq, Els, xhat

```

```

[ ]: def pade(x, p, q):
    N_sig = len(x)
    if p > 0:
        # 1. Résoudre pour ap
        X_f = get_conv_mtx(x, p)
        Xq = X_f[q : q+p, 0:p]
        # Vérification de singularité
        if np.linalg.det(Xq) == 0:
            raise ValueError("Xq est une matrice singulière !")

        xq_1 = x[q+1 : q+p+1]
        ap_t = solve(Xq, -xq_1)
        ap = np.concatenate(([1.0], ap_t))
    else:
        ap = np.array([1.0])

    # 2. Résoudre pour bq
    X_f = get_conv_mtx(x, p)
    X_top = X_f[0:q+1, 0:p+1]
    bq = X_top @ ap

    # 3. Reconstruction
    xhat = lfilter(bq, ap, np.eye(1, N_sig)[0])
    Els = np.linalg.norm(x - xhat)

    return ap, bq, Els, xhat

```


1.6 4.1.6 Exemple: Design de filtre

1.6.1 4.1.6.1 Définition du filtre idéal

L'objectif est de synthétiser un filtre à partir de contraintes fréquentielles. On souhaite un filtre passe-bas idéal avec une phase linéaire (retard pur) :

$$H(jf) = \begin{cases} e^{-jn_d 2\pi f} & \text{pour } |f| < F_p \\ 0 & \text{pour } F_p < |f| < 0.5 \end{cases}$$

Puisque la réponse en fréquence est une fonction rectangulaire, la réponse impulsionnelle désirée $h(n)$ est une fonction **sinc** décalée de n_d échantillons pour assurer la causalité :

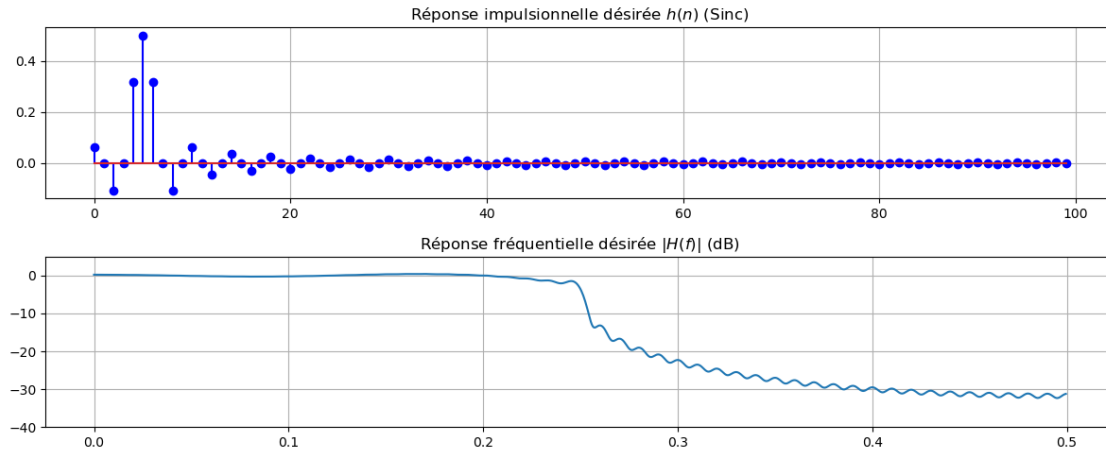
$$h(n) = 2F_p \text{sinc}(2F_p(n - n_d)) = \frac{\sin((n - n_d)\pi/2)}{(n - n_d)\pi}$$

$N = 100, F_p = 0.25, n_d = 5$. Approximation avec un modèle ARMA(6,6).

```
[ ]: N_filt = 100
Fp = 0.25
nd = 5
n_filt = np.arange(N_filt)
hn = 0.5 * np.sinc((n_filt - nd) * 2 * Fp)

# Visualisation du filtre idéal
plt.figure(figsize=(12, 5))
plt.subplot(2, 1, 1)
plt.stem(n_filt, hn, linefmt='b-', markerfmt='bo')
plt.title("Réponse impulsionnelle désirée $h(n)$ (Sinc)")
plt.grid(True)

plt.subplot(2, 1, 2)
w, H_ideal = freqz(hn, 1, worN=1024)
plt.plot(w/(2*np.pi), 20*np.log10(np.abs(H_ideal)))
plt.title("Réponse fréquentielle désirée $|H(f)|$ (dB)")
plt.ylim([-40, 5])
plt.grid(True)
plt.tight_layout()
plt.show()
```



1.6.2 4.1.6.2 Synthèse avec fonction Padé ordre $p=6$, $q=6$

Utiliser la fonction Padé pour synthétiser le filtre $H(z)$ en produisant une réponse impulsionnelle comme celle du filtre voulu.

```
[ ]: # Approximation Padé ARMA(6,6)
p_iir, q_iir = 6, 6
ap_iir, bq_iir, els_iir, hhat_iir = pade(hn, p_iir, q_iir)

# Affichage fréquentiel
w, H_ideal = freqz(hn, 1, worN=1024)
w, H_pade = freqz(bq_iir, ap_iir, worN=1024)

plt.figure(figsize=(10, 6))
plt.plot(w/(2*np.pi), 20*np.log10(np.abs(H_ideal)), 'b', label='Désiré (Sinc)')
plt.plot(w/(2*np.pi), 20*np.log10(np.abs(H_pade)), 'r--', label='Padé_
↳ARMA(6,6)')
plt.title('Réponse Fréquentielle : Filtre Idéal vs Padé')
plt.xlabel('Fréquence normalisée')
plt.ylabel('Magnitude (dB)')
plt.ylim([-50, 5])
plt.legend()
plt.grid(True)
plt.show()
```

